

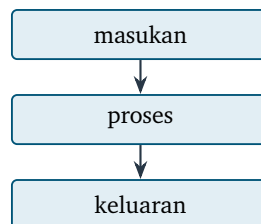
4

Percabangan dan Pengulangan

4.1 Aliran kontrol dalam program

Pada kuliah sebelumnya kita telah mempelajari bagaimana algoritma diterjemahkan menjadi program C menggunakan variabel, tipe data, ekspresi, operator relasional, operator logika, serta input dan output.

Program-program tersebut sebagian besar masih berjalan secara berurutan:



Namun, banyak persoalan rekayasa tidak dapat diselesaikan hanya dengan urutan lurus. Program sering perlu mengambil keputusan.

Sebagai contoh:

- ▶ jika temperatur terlalu tinggi, aktifkan peringatan;
- ▶ jika tekanan berada di luar rentang aman, tampilkan alarm;
- ▶ jika input tidak valid, minta pengguna memasukkan nilai kembali.

Program juga sering perlu mengulangi proses.

Sebagai contoh:

- ▶ membaca 100 data sensor;
- ▶ menghitung posisi benda untuk beberapa nilai waktu;

- ▶ mengulangi pengukuran sampai kondisi tertentu tercapai;
- ▶ menghitung jumlah, rata-rata, minimum, dan maksimum dari sekumpulan data.

Dua kemampuan tersebut disebut **percabangan** (*branching*) dan **pengulangan** (*looping* atau *iteration*).

Secara konseptual, program dapat dibangun dari tiga pola dasar:

```
urutan
percabangan
pengulangan
```

Ketiganya merupakan fondasi utama aliran kontrol dalam pemrograman terstruktur.

4.2 Percabangan

Percabangan memungkinkan program memilih jalur eksekusi berdasarkan suatu kondisi.

Secara algoritmik,

```
IF kondisi
    lakukan sesuatu
END IF
```

Dalam C, pola tersebut ditulis menggunakan **if**.

4.2.1 Bentuk dasar **if**

Contoh paling sederhana:

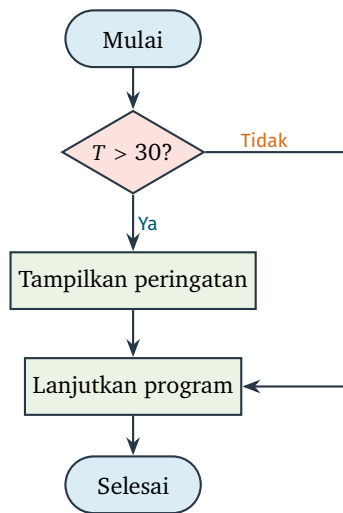
```
if (temperature > 30.0)
{
    printf("Temperature is high.\n");
}
```

Bagian `temperature > 30.0` merupakan ekspresi Boolean.

Jika kondisi bernilai benar, blok di dalam kurung kurawal dijalankan.

Jika kondisi bernilai salah, blok tersebut dilewati.

Bentuk diagram alirnya ditunjukkan di bawah ini.



4.2.2 Contoh: peringatan temperatur

Misalkan sebuah sistem termal memberikan peringatan jika

$$T > 80^{\circ}\text{C}.$$

Program C:

```
#include <stdio.h>

int main(void)
{
    double temperature;

    printf("Temperature (C): ");
    scanf("%lf", &temperature);

    if (temperature > 80.0)
    {
        printf("WARNING: temperature too high.\n");
    }

    printf("Measurement complete.\n");

    return 0;
}
```

Jika pengguna memasukkan 85 program menampilkan

```
WARNING: temperature too high.
Measurement complete.
```

Jika pengguna memasukkan 60 program hanya menampilkan Measurement complete..

Perhatikan bahwa **if** tidak menghentikan program. **if** hanya mengatur apakah suatu blok dijalankan atau dilewati.

4.3 Percabangan dua arah: **if** dan **else**

Sering kali kita ingin memilih antara dua kemungkinan.

Secara algoritmik,

```
IF kondisi
    lakukan A
ELSE
    lakukan B
END IF
```

Dalam C,

```
if (condition)
{
    /* A */
}
else
{
    /* B */
}
```

4.3.1 Contoh: status sensor

Misalkan sensor dianggap aman jika

$$20^{\circ}\text{C} \leq T \leq 30^{\circ}\text{C}.$$

Kondisi tersebut dalam C adalah

```
temperature >= 20.0 &&
temperature <= 30.0
```

Program:

```
#include <stdio.h>

int main(void)
{
    double temperature;

    printf("Temperature (C): ");
    scanf("%lf", &temperature);

    if (temperature >= 20.0 &&
        temperature <= 30.0)
    {
        printf("Status: NORMAL\n");
    }
}
```

```

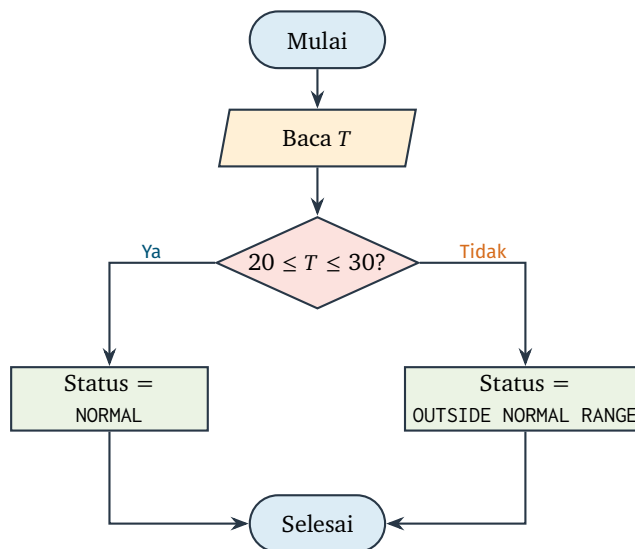
    }
    else
    {
        printf("Status: OUTSIDE NORMAL RANGE\n");
    }

    return 0;
}

```

Di sini salah satu dari dua blok pasti dijalankan.

4.3.2 Diagram alir



4.4 Percabangan banyak arah: `else if`

Persoalan rekayasa sering memiliki lebih dari dua kategori.

Misalkan temperatur diklasifikasikan sebagai

DINGIN	jika $T < 20$
NORMAL	jika $20 \leq T \leq 30$
PANAS	jika $T > 30$

Dalam C:

```

#include <stdio.h>

int main(void)
{
    double temperature;

    printf("Temperature (C): ");

```

```

scanf("%lf", &temperature);

if (temperature < 20.0)
{
    printf("Status: DINGIN\n");
}
else if (temperature <= 30.0)
{
    printf("Status: NORMAL\n");
}
else
{
    printf("Status: PANAS\n");
}

return 0;
}

```

Perhatikan bahwa pada blok kedua kita cukup menulis `temperature <= 30.0` bukan

```

temperature >= 20.0 &&
temperature <= 30.0

```

Mengapa?

Karena jika program sudah mencapai `else if`, kondisi `temperature < 20.0` telah diketahui salah. Dengan demikian secara otomatis

$$T \geq 20.$$

Kondisi kedua hanya perlu memeriksa batas atas.

4.4.1 Urutan kondisi sangat penting

Perhatikan program berikut.

```

if (temperature <= 30.0)
{
    printf("Not hot\n");
}
else if (temperature < 20.0)
{
    printf("Cold\n");
}

```

Blok kedua tidak mungkin dijalankan.

Jika

$$T < 20,$$

maka

$$T \leq 30$$

juga benar sehingga program sudah masuk ke blok pertama.

Karena itu, pada rantai `if-else if`, kondisi harus disusun secara logis.

4.5 Contoh fisika: klasifikasi bilangan Reynolds

Bilangan Reynolds dalam mekanika fluida diberikan oleh

$$Re = \frac{\rho v L}{\mu}.$$

Sebagai ilustrasi sederhana, kita gunakan klasifikasi

laminar	jika $Re < 2300$
transisi	jika $2300 \leq Re < 4000$
turbulen	jika $Re \geq 4000$

Catatan: nilai batas tersebut merupakan penyederhanaan untuk ilustrasi dan bergantung pada konfigurasi aliran.

Program:

```
#include <stdio.h>

int main(void)
{
    double density;
    double velocity;
    double length;
    double viscosity;

    printf("Density (kg/m^3): ");
    scanf("%lf", &density);

    printf("Velocity (m/s): ");
    scanf("%lf", &velocity);

    printf("Characteristic length (m): ");
    scanf("%lf", &length);

    printf("Dynamic viscosity (Pa s): ");
    scanf("%lf", &viscosity);

    double reynolds =
        density * velocity * length / viscosity;

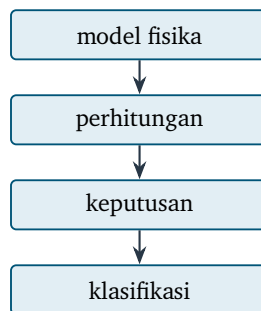
    printf("Re = %.3f\n", reynolds);

    if (reynolds < 2300.0)
    {
        printf("Flow regime: laminar\n");
    }
}
```

```
else if (reynolds < 4000.0)
{
    printf("Flow regime: transition\n");
}
else
{
    printf("Flow regime: turbulent\n");
}

return 0;
}
```

Program tersebut memperlihatkan hubungan



4.6 Percabangan bertingkat

Sebuah **if** dapat berada di dalam **if** lain. Struktur seperti ini disebut **nested if** atau percabangan bertingkat.

Contoh:

```
if (sensor_active)
{
    if (temperature > 80.0)
    {
        printf("ALARM\n");
    }
}
```

Secara logika, alarm hanya aktif jika

$$\text{sensor aktif} \wedge T > 80.$$

Kita juga dapat menulis kondisi tersebut sebagai

```
if (sensor_active &&
    temperature > 80.0)
{
    printf("ALARM\n");
}
```

Kedua pendekatan dapat menghasilkan perilaku yang sama. Pilihan terbaik bergantung pada struktur logika yang ingin kita tekankan.

4.6.1 Hindari percabangan terlalu dalam

Percabangan bertingkat yang terlalu dalam membuat kode sulit dibaca.

Contoh:

```
if (a)
{
    if (b)
    {
        if (c)
        {
            if (d)
            {
                /* ... */
            }
        }
    }
}
```

Jika kondisi dapat digabungkan secara wajar,

```
if (a && b && c && d)
{
    /* ... */
}
```

sering lebih mudah dipahami.

Namun, tidak semua percabangan bertingkat dapat atau sebaiknya digabungkan.

4.7 switch

C menyediakan struktur **switch** untuk memilih satu dari beberapa kasus berdasarkan nilai diskrit.

Bentuk umum:

```
switch (expression)
{
    case value1:
        /* statements */
        break;

    case value2:
        /* statements */
        break;

    default:
```

```
    /* statements */  
    break;  
}
```

`switch` cocok ketika keputusan didasarkan pada satu nilai integer atau karakter yang memiliki sejumlah pilihan diskrit.

4.7.1 Contoh: mode operasi

Misalkan suatu perangkat mempunyai mode

```
1 = standby  
2 = measurement  
3 = calibration
```

Program:

```
#include <stdio.h>  
  
int main(void)  
{  
    int mode;  
  
    printf("Mode (1-3): ");  
    scanf("%d", &mode);  
  
    switch (mode)  
    {  
        case 1:  
            printf("STANDBY mode\n");  
            break;  
  
        case 2:  
            printf("MEASUREMENT mode\n");  
            break;  
  
        case 3:  
            printf("CALIBRATION mode\n");  
            break;  
  
        default:  
            printf("Invalid mode\n");  
            break;  
    }  
  
    return 0;  
}
```

4.7.2 Fungsi break

Tanpa **break**, eksekusi dapat berlanjut ke kasus berikutnya.

Perhatikan:

```
switch (mode)
{
    case 1:
        printf("Mode 1\n");

    case 2:
        printf("Mode 2\n");
        break;
}
```

Jika

```
mode = 1
```

program dapat mencetak

```
Mode 1
Mode 2
```

Perilaku tersebut disebut **fall-through**.

Kadang-kadang *fall-through* digunakan secara sengaja, tetapi untuk pemula sebaiknya gunakan **break** secara eksplisit kecuali memang ada alasan kuat untuk tidak melakukannya.

4.8 if atau switch?

Gunakan **if** ketika kondisi berbentuk

- ▶ rentang;
- ▶ perbandingan;
- ▶ kombinasi logika;
- ▶ ekspresi Boolean kompleks.

Contoh:

```
if (temperature > 80.0)
```

atau

```
if (pressure >= min_pressure &&
    pressure <= max_pressure)
```

Gunakan **switch** ketika keputusan didasarkan pada satu nilai diskrit.

Contoh:

switch (mode)

dengan beberapa nilai seperti

```
1
2
3
```

4.9 Operator ternary

C memiliki operator kondisi singkat `?:`.

Bentuk umum:

```
condition ? value_if_true : value_if_false
```

Contoh:

```
int max =
    (a > b) ? a : b;
```

Ekspresi tersebut memilih nilai `a` jika `a > b`, selain itu memilih `b`.

Operator ternary berguna untuk ekspresi pendek. Namun, jika logika mulai kompleks, gunakan `if` agar kode tetap mudah dibaca.

4.10 Kontrol percabangan vs. pengulangan

Percabangan memungkinkan program memilih jalur.

Pengulangan memungkinkan program menjalankan suatu blok berkali-kali.

Secara umum, pengulangan memerlukan tiga hal:

1. keadaan awal;
2. kondisi untuk melanjutkan;
3. perubahan keadaan.

Sebagai contoh, jika kita ingin mencetak angka 1 sampai 5:

```
i ← 1
WHILE i ≤ 5
    OUTPUT i
    i ← i + 1
END WHILE
```

Nilai `i` berubah pada setiap iterasi.

Ketika

$i = 6$,

kondisi

$$i \leq 5$$

menjadi salah sehingga pengulangan berhenti.

4.11 while

Bentuk umum:

```
while (condition)
{
    /* statements */
}
```

Kondisi diperiksa **sebelum** blok dijalankan. Karena itu, blok dapat tidak dijalankan sama sekali.

4.11.1 Contoh dasar

```
#include <stdio.h>

int main(void)
{
    int i = 1;

    while (i <= 5)
    {
        printf("%d\n", i);

        i = i + 1;
    }

    return 0;
}
```

Keluaran:

```
1
2
3
4
5
```

4.11.2 Tracing

Mari kita telusuri.

Iterasi	i sebelum pemeriksaan	Kondisi i <= 5	Output	i setelah pembaruan
1	1	true	1	2
2	2	true	2	3
3	3	true	3	4
4	4	true	4	5
5	5	true	5	6
akhir	6	false	-	-

Tabel seperti ini membantu kita memahami perubahan keadaan program.

4.12 Pengulangan untuk perhitungan fisika

Misalkan kita ingin menghitung posisi benda untuk waktu

$$t = 0, 1, 2, 3, 4, 5 \text{ s}$$

dengan

$$x(t) = x_0 + v_0 t + \frac{1}{2} a t^2.$$

Program:

```
#include <stdio.h>

int main(void)
{
    const double x0 = 0.0;
    const double v0 = 5.0;
    const double acceleration = 2.0;

    int t = 0;

    while (t <= 5)
    {
        double position =
            x0
            + v0 * t
            + 0.5 * acceleration * t * t;

        printf("t = %d s, x = %.3f m\n",
            t, position);

        t = t + 1;
    }

    return 0;
}
```

Di sini pengulangan digunakan untuk mengevaluasi model fisika pada beberapa titik waktu.

4.12.1 Akumulator

Salah satu pola pengulangan terpenting adalah **accumulator**.

Misalkan kita ingin menghitung

$$S = \sum_{i=1}^N x_i.$$

Pseudocode:

```
sum ← 0
FOR setiap data x
    sum ← sum + x
END FOR
```

Dalam C kita dapat menggunakan **while**:

```
#include <stdio.h>

int main(void)
{
    int n;

    printf("Number of data: ");
    scanf("%d", &n);

    int i = 1;
    double sum = 0.0;

    while (i <= n)
    {
        double x;

        printf("x%d: ", i);
        scanf("%Lf", &x);

        sum = sum + x;

        i = i + 1;
    }

    printf("Sum = %.3f\n", sum);

    return 0;
}
```

Variabel **sum** disebut akumulator karena menyimpan hasil yang terus diperbarui.

4.12.2 Counter

Pola lain adalah **counter**.

Misalkan kita ingin menghitung berapa banyak data temperatur yang memenuhi

$$T > 30^{\circ}\text{C}.$$

```
int count_high = 0;

if (temperature > 30.0)
{
    count_high = count_high + 1;
}
```

Di dalam pengulangan, pola tersebut dapat digunakan untuk menghitung jumlah kejadian.

4.13 do-while

Bentuk umum:

```
do
{
    /* statements */
}
while (condition);
```

Perbedaan penting dengan **while** adalah bahwa blok **do-while** dijalankan **setidaknya satu kali** sebelum kondisi diperiksa.

4.13.1 Contoh

```
#include <stdio.h>

int main(void)
{
    int value;

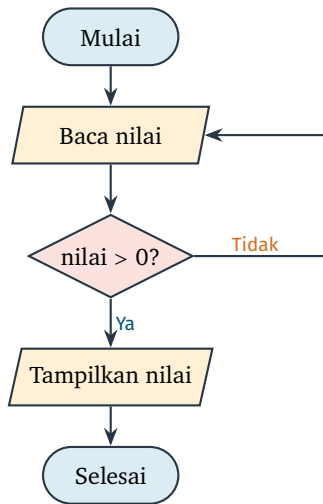
    do
    {
        printf("Enter a positive integer: ");
        scanf("%d", &value);
    }
    while (value <= 0);

    printf("Accepted value = %d\n", value);

    return 0;
}
```

Jika pengguna memasukkan nilai tidak valid, program meminta kembali.

4.13.2 Diagram alir



4.13.3 Kapan do-while sesuai?

do-while cocok ketika suatu tindakan memang harus dilakukan minimal satu kali.

Contoh:

- ▶ menampilkan menu;
- ▶ meminta input;
- ▶ melakukan satu langkah perhitungan awal;
- ▶ membaca data sebelum memutuskan apakah perlu mengulang.

4.14 for

Struktur **for** banyak digunakan ketika pola pengulangan memiliki

- ▶ nilai awal;
- ▶ kondisi;
- ▶ pembaruan

yang jelas.

Bentuk umum:

```

for (initialization; condition; update)
{
    /* statements */
}
  
```

Contoh:

```
for (int i = 1; i <= 5; i++)
{
    printf("%d\n", i);
}
```

Secara algoritmik setara dengan

```
i ← 1
WHILE i ≤ 5
    OUTPUT i
    i ← i + 1
END WHILE
```

4.14.1 Bagian for

Pada

```
for (int i = 1; i <= 5; i++)
```

bagian

```
int i = 1
```

merupakan inisialisasi.

Bagian `i <= 5` merupakan kondisi.

Bagian `i++` merupakan pembaruan.

Operator `i++` setara secara konseptual dengan

```
i = i + 1;
```

untuk penggunaan sederhana seperti ini.

4.14.2 for untuk tabulasi perhitungan

Misalkan kita ingin menghitung

$$x(t) = x_0 + v_0 t + \frac{1}{2} a t^2$$

untuk

$$t = 0, 1, \dots, 10.$$

Program:

```
#include <stdio.h>

int main(void)
{
    const double x0 = 2.0;
    const double v0 = 5.0;
    const double acceleration = 3.0;
```

```
for (int t = 0; t <= 10; t++)
{
    double position =
        x0
        + v0 * t
        + 0.5 * acceleration * t * t;

    printf("%d %.3f\n", t, position);
}

return 0;
}
```

Keluaran dapat digunakan untuk membuat tabel $t(s)$ $x(m)$ yang nantinya dapat disimpan atau divisualisasikan.

4.15 `while` atau `for`?

Secara umum, `for` cocok jika jumlah iterasi atau pola pencacah (*counter*) sudah jelas.

Contoh:

```
for (int i = 0; i < 100; i++)
```

`while` cocok jika pengulangan lebih alami dinyatakan melalui suatu kondisi.

Contoh:

```
while (error > tolerance)
```

Keduanya secara teori dapat saling menggantikan pada banyak kasus, tetapi pilihan yang tepat membuat maksud program lebih jelas.

4.16 Pengulangan tak hingga

Kesalahan umum adalah lupa memperbarui variabel yang memengaruhi kondisi.

Contoh:

```
int i = 1;

while (i <= 5)
{
    printf("%d\n", i);
}
```

Nilai `i` tidak pernah berubah.

Kondisi `i <= 5` selalu benar.

Program dapat berjalan tanpa akhir.

Perbaikannya adalah memperbarui **i**:

```
i = i + 1;
```

Setiap pengulangan berbasis kondisi harus diperiksa:

Apa yang berubah pada setiap iterasi dan bagaimana perubahan tersebut membuat kondisi berhenti akhirnya tercapai?

4.17 Off-by-one error

Kesalahan lain yang sangat umum adalah jumlah iterasi lebih satu atau kurang satu.

Misalkan kita ingin memproses N data dengan indeks $0, 1, \dots, N - 1$.

Pengulangan yang sesuai adalah

```
for (int i = 0; i < n; i++)
```

bukan

```
for (int i = 0; i <= n; i++)
```

Versi kedua berjalan $N + 1$ kali.

Kesalahan semacam ini disebut **off-by-one error**.

4.18 break

Kata kunci

```
break;
```

dapat digunakan untuk keluar dari pengulangan sebelum kondisi normal selesai.

Contoh:

```
for (int i = 1; i <= 100; i++)  
{  
    if (i == 10)  
    {  
        break;  
    }  
  
    printf("%d\n", i);  
}
```

Program hanya mencetak

```
1  
2  
...
```

9

karena pengulangan berhenti ketika `i == 10`.

4.18.1 Penggunaan dalam pencarian sederhana

Misalkan kita membaca nilai sampai ditemukan nilai negatif.

```
#include <stdio.h>

int main(void)
{
    for (int i = 1; i <= 100; i++)
    {
        double x;

        printf("x%d: ", i);
        scanf("%lf", &x);

        if (x < 0.0)
        {
            printf("Negative value found.\n");
            break;
        }
    }

    return 0;
}
```

`break` berguna, tetapi penggunaan berlebihan dapat membuat aliran program lebih sulit dipahami.

4.19 continue

Kata kunci

```
continue;
```

melewati sisa blok pada iterasi saat ini dan melanjutkan ke iterasi berikutnya.

Contoh:

```
for (int i = 1; i <= 10; i++)
{
    if (i % 2 != 0)
    {
        continue;
    }

    printf("%d\n", i);
}
```

Program hanya mencetak bilangan genap.

Untuk program awal, `continue` sebaiknya digunakan secara terbatas agar aliran program tetap mudah ditelusuri.

4.20 Pengulangan bersarang

Pengulangan dapat ditempatkan di dalam pengulangan lain.

Contoh:

```
for (int i = 1; i <= 3; i++)
{
    for (int j = 1; j <= 4; j++)
    {
        printf("i = %d, j = %d\n", i, j);
    }
}
```

Pengulangan luar dilakukan tiga kali.

Pada setiap iterasi luar, pengulangan dalam dilakukan empat kali.

Total kombinasi:

$$3 \times 4 = 12.$$

4.20.1 Kompleksitas intuitif

Jika kedua batas sama-sama sebesar N , maka jumlah iterasi adalah kira-kira N^2 .

Karena itu, pengulangan bersarang sering berkaitan dengan kompleksitas $O(N^2)$.

Konsep ini menghubungkan langsung materi Kuliah 2 dengan implementasi C.

4.21 Contoh fisika: tabel gaya pegas

Hukum Hooke:

$$F = -kx.$$

Misalkan kita ingin membuat tabel gaya untuk beberapa nilai perpindahan.

```
#include <stdio.h>

int main(void)
{
    const double k = 100.0;

    for (int i = -5; i <= 5; i++)
    {
        double x = 0.01 * i;
        double force = -k * x;

        printf("x = %.3f m, F = %.3f N\n",
```

```
        x, force);  
    }  
  
    return 0;  
}
```

Program tersebut menghitung gaya untuk

$$x = -0.05, -0.04, \dots, 0.05 \text{ m.}$$

Pengulangan membuat kita tidak perlu menulis sebelas ekspresi terpisah.

4.22 Contoh fisika: klasifikasi sensor berulang

Misalkan kita ingin membaca lima temperatur dan mengklasifikasikannya.

```
#include <stdio.h>  
  
int main(void)  
{  
    for (int i = 1; i <= 5; i++)  
    {  
        double temperature;  
  
        printf("T%d (C): ", i);  
        scanf("%lf", &temperature);  
  
        if (temperature < 20.0)  
        {  
            printf("DINGIN\n");  
        }  
        else if (temperature <= 30.0)  
        {  
            printf("NORMAL\n");  
        }  
        else  
        {  
            printf("PANAS\n");  
        }  
    }  
  
    return 0;  
}
```

Di sini percabangan berada di dalam pengulangan.

Artinya, keputusan dilakukan untuk setiap data.

4.23 Contoh: menghitung rata-rata

Misalkan kita membaca N data.

Rata-ratanya adalah

$$\bar{x} = \frac{1}{N} \sum_{i=1}^N x_i.$$

Implementasi:

```
#include <stdio.h>

int main(void)
{
    int n;

    printf("Number of data: ");
    scanf("%d", &n);

    if (n <= 0)
    {
        printf("Number of data must be positive.\n");
        return 1;
    }

    double sum = 0.0;

    for (int i = 1; i <= n; i++)
    {
        double x;

        printf("x%d: ", i);
        scanf("%lf", &x);

        sum = sum + x;
    }

    double mean = sum / n;

    printf("Mean = %.3f\n", mean);

    return 0;
}
```

4.23.1 Tracing

Untuk data 2, 4, 6, keadaan `sum` berubah sebagai berikut.

Iterasi	x_i	sum sebelum	sum sesudah
1	2	0	2
2	4	2	6
3	6	6	12

Kemudian,

$$\bar{x} = \frac{12}{3} = 4.$$

4.24 Contoh: minimum dan maksimum

Pola minimum dan maksimum membutuhkan inisialisasi yang benar.

Misalkan data pertama dibaca terlebih dahulu.

```
#include <stdio.h>

int main(void)
{
    int n;

    printf("Number of data: ");
    scanf("%d", &n);

    if (n <= 0)
    {
        printf("Number of data must be positive.\n");
        return 1;
    }

    double x;

    printf("x1: ");
    scanf("%lf", &x);

    double minimum = x;
    double maximum = x;

    for (int i = 2; i <= n; i++)
    {
        printf("x%d: ", i);
        scanf("%lf", &x);

        if (x < minimum)
        {
            minimum = x;
        }

        if (x > maximum)
        {
            maximum = x;
        }
    }

    printf("Minimum = %.3f\n", minimum);
    printf("Maximum = %.3f\n", maximum);
}
```

```
    return 0;  
}
```

Menginisialisasi

```
minimum = 0.0;  
maximum = 0.0;
```

tidak selalu benar.

Contoh data -5 , -2 , -8 akan menghasilkan maksimum yang salah jika `maximum` dimulai dari nol.

4.25 Studi kasus monitoring temperatur

Kita sekarang menggabungkan

- ▶ input;
- ▶ percabangan;
- ▶ pengulangan;
- ▶ akumulator;
- ▶ counter;
- ▶ minimum dan maksimum.

4.25.1 Spesifikasi

Masukan:

$$N, \quad T_1, T_2, \dots, T_N$$

dan ambang T_{limit} .

Keluaran:

- ▶ rata-rata;
- ▶ minimum;
- ▶ maksimum;
- ▶ jumlah data di atas ambang.

4.25.2 Pseudocode

```
INPUT N  
INPUT  $T_{\text{limit}}$   
IF  $N \leq 0$   
    STOP  
END IF  
INPUT  $T_1$   
 $sum \leftarrow T_1$ 
```

```

minimum  $\leftarrow T_1$ 
maximum  $\leftarrow T_1$ 
IF  $T_1 > T_{\text{limit}}$ 
    count_high  $\leftarrow 1$ 
ELSE
    count_high  $\leftarrow 0$ 
END IF
FOR  $i \leftarrow 2$  TO  $N$ 
    INPUT  $T_i$ 
     $\text{sum} \leftarrow \text{sum} + T_i$ 
    IF  $T_i < \text{minimum}$ 
        minimum  $\leftarrow T_i$ 
    END IF
    IF  $T_i > \text{maximum}$ 
        maximum  $\leftarrow T_i$ 
    END IF
    IF  $T_i > T_{\text{limit}}$ 
        count_high  $\leftarrow \text{count\_high} + 1$ 
    END IF
END FOR
mean  $\leftarrow \text{sum}/N$ 
OUTPUT mean
OUTPUT minimum
OUTPUT maximum
OUTPUT count_high

```

4.25.3 Implementasi C

```

#include <stdio.h>

int main(void)
{
    int n;
    double limit;

    printf("Number of measurements: ");
    scanf("%d", &n);

    printf("Temperature limit (C): ");
    scanf("%Lf", &limit);

    if (n <= 0)
    {
        printf("Number of measurements must be positive.\n");
        return 1;
    }

    double temperature;

    printf("T1: ");

```

```
scanf("%lf", &temperature);

double sum = temperature;
double minimum = temperature;
double maximum = temperature;

int count_high = 0;

if (temperature > limit)
{
    count_high = 1;
}

for (int i = 2; i <= n; i++)
{
    printf("T%d: ", i);
    scanf("%lf", &temperature);

    sum = sum + temperature;

    if (temperature < minimum)
    {
        minimum = temperature;
    }

    if (temperature > maximum)
    {
        maximum = temperature;
    }

    if (temperature > limit)
    {
        count_high = count_high + 1;
    }
}

double mean = sum / n;

printf("\nMean      = %.3f C\n", mean);
printf("Minimum    = %.3f C\n", minimum);
printf("Maximum     = %.3f C\n", maximum);
printf("Above limit = %d\n", count_high);

return 0;
}
```

4.25.4 Kompleksitas

Setiap data diproses satu kali.

Karena itu, kompleksitas waktu adalah $O(N)$.

Program hanya menggunakan sejumlah kecil variabel tambahan yang tidak bertambah terhadap N .

Karena itu, kebutuhan memori tambahannya secara intuitif adalah $O(1)$.

4.26 Masalah validasi input

Program teknik perlu memeriksa apakah input masuk akal.

Misalkan fraksi efisiensi harus memenuhi

$$0 \leq \eta \leq 1.$$

Kita ingin terus meminta input sampai nilai valid diberikan.

```
#include <stdio.h>

int main(void)
{
    double efficiency;

    do
    {
        printf("Efficiency [0, 1]: ");
        scanf("%lf", &efficiency);

        if (efficiency < 0.0 ||
            efficiency > 1.0)
        {
            printf("Invalid value.\n");
        }
    }
    while (efficiency < 0.0 ||
          efficiency > 1.0);

    printf("Accepted efficiency = %.3f\n",
          efficiency);

    return 0;
}
```

Program ini menunjukkan kombinasi

`do-while` + `if` + operator logika

4.27 Studi kasus: pendekatan akar secara iteratif

Pengulangan tidak hanya digunakan untuk memproses data.

Pengulangan juga dapat digunakan untuk algoritma iteratif.

Misalkan kita ingin mencari secara sederhana nilai x yang memenuhi

$$x^2 \approx 2.$$

Kita dapat memulai dari

$$x = 0$$

dan menambah

$$\Delta x = 0.001$$

sampai

$$x^2 \geq 2.$$

Program:

```
#include <stdio.h>

int main(void)
{
    const double target = 2.0;
    const double dx = 0.001;

    double x = 0.0;

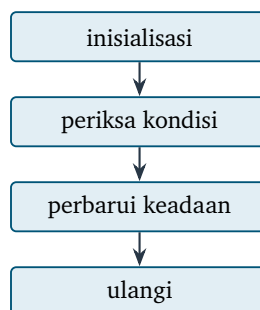
    while (x * x < target)
    {
        x = x + dx;
    }

    printf("Approximate sqrt(2) = %.6f\n", x);

    return 0;
}
```

Ini bukan metode paling efisien untuk menghitung akar kuadrat.

Namun, contoh tersebut memperlihatkan pola penting:



Pola seperti ini akan muncul kembali pada algoritma numerik.

4.28 Studi kasus: jumlah deret

Pertimbangkan

$$S_N = \sum_{n=1}^N \frac{1}{n^2}.$$

Program:

```
#include <stdio.h>

int main(void)
{
    int n_max;

    printf("N: ");
    scanf("%d", &n_max);

    if (n_max <= 0)
    {
        printf("N must be positive.\n");
        return 1;
    }

    double sum = 0.0;

    for (int n = 1; n <= n_max; n++)
    {
        double term =
            1.0 / ((double)n * n);

        sum = sum + term;
    }

    printf("S_N = %.12f\n", sum);

    return 0;
}
```

Secara matematis,

$$\lim_{N \rightarrow \infty} \sum_{n=1}^N \frac{1}{n^2} = \frac{\pi^2}{6}.$$

Kita belum mempelajari metode numerik secara formal, tetapi contoh ini menunjukkan bagaimana pengulangan digunakan untuk membentuk pendekatan terhadap suatu limit.

4.29 Pengulangan dan keadaan program

Pada setiap iterasi, program mempunyai suatu keadaan.

Contoh:

```
sum = sum + x;
count = count + 1;
```

mengubah keadaan program.

Setelah memproses i data, kita dapat memiliki hubungan

$$\text{sum} = \sum_{k=1}^i x_k$$

dan

$$\text{count} = i.$$

Hubungan seperti ini merupakan bentuk intuitif dari **invarian** yang telah diperkenalkan pada Kuliah 2.

Memahami keadaan program membantu kita melakukan *tracing* dan *debugging*.

4.30 Kesalahan umum dalam percabangan

4.30.1 Menggunakan `=` sebagai pengganti `==`

Salah:

```
if (mode = 1)
```

Ini melakukan assignment.

Yang dimaksud biasanya:

```
if (mode == 1)
```

4.30.2 Kondisi rentang yang salah

Salah:

```
if (20.0 <= temperature <= 30.0)
```

Benar:

```
if (temperature >= 20.0 &&
    temperature <= 30.0)
```

4.30.3 Kondisi tumpang tindih atau tidak lengkap

Perhatikan:

```
if (temperature < 20.0)
{
    /* cold */
}
else if (temperature > 30.0)
```

```
{  
    /* hot */  
}
```

Bagaimana jika

$$20 \leq T \leq 30?$$

Tidak ada keluaran.

Pastikan semua kasus yang relevan ditangani.

4.30.4 Salah mengurutkan kondisi

Kondisi yang terlalu umum dapat menutupi kondisi yang lebih khusus.

4.31 Kesalahan umum dalam pengulangan

4.31.1 Tidak memperbarui pencacah

```
while (i <= n)  
{  
    printf("%d\n", i);  
}
```

menghasilkan pengulangan tak hingga.

4.31.2 Pembaruan ke arah yang salah

```
int i = 1;  
  
while (i <= 10)  
{  
    i--;  
}
```

Nilai `i` justru semakin kecil sehingga kondisi tetap benar.

4.31.3 Off-by-one

```
for (int i = 0; i <= n; i++)
```

berjalan sebanyak $N + 1$ iterasi jika dimulai dari nol.

4.31.4 Mengubah variabel kontrol secara tidak sengaja

```
for (int i = 0; i < n; i++)  
{  
    i = i + 2;  
}
```

Pembaruan `i` terjadi dua kali.

Hal ini dapat membuat pola iterasi sulit dipahami.

4.31.5 Kondisi berhenti pada *floating-point*

Dalam komputasi *floating-point*, kondisi seperti `while (x != 1.0)` dapat bermasalah jika `x` diperbarui menggunakan operasi numerik yang tidak pernah menghasilkan `1.0` secara eksak.

Lebih aman menggunakan toleransi jika konteksnya numerik.

```
while (fabs(x - 1.0) > tolerance)
```

4.32 Kebiasaan pemrograman yang baik untuk aliran kontrol

- ▶ Gunakan kurung kurawal meskipun blok hanya satu baris.

Lebih baik:

```
if (x > 0)
{
    printf("positive\n");
}
```

daripada

```
if (x > 0)
    printf("positive\n");
```

- ▶ Gunakan indentasi konsisten.
- ▶ Hindari percabangan bertingkat terlalu dalam.
- ▶ Gunakan nama kondisi yang bermakna jika ekspresi terlalu panjang.

Contoh:

```
bool safe_temperature =
    temperature >= 20.0 &&
    temperature <= 30.0;

if (safe_temperature)
{
    printf("NORMAL\n");
}
```

- ▶ Pastikan pengulangan memiliki mekanisme berhenti.
- ▶ Lakukan *tracing* untuk kasus sederhana.
- ▶ Uji nilai batas.

Misalnya jika klasifikasi berubah pada

$$T = 20$$

dan

$$T = 30,$$

uji tepat pada kedua batas tersebut.

- ▶ Kompilasi dengan peringatan aktif.

```
gcc -Wall -Wextra -Wpedantic -std=c17 program.c -o program
```

4.33 Cek pemahaman

Cobalah menjawab pertanyaan berikut tanpa melihat kembali catatan.

- ▶ Apa yang dimaksud dengan aliran kontrol?
- ▶ Apa tiga pola dasar pemrograman terstruktur?
- ▶ Apa perbedaan `if`, `if-else`, dan `if-else if-else`?
- ▶ Mengapa urutan kondisi dalam rantai `else if` penting?
- ▶ Kapan `switch` lebih sesuai daripada `if`?
- ▶ Apa fungsi `break` dalam `switch`?
- ▶ Apa yang dimaksud dengan *fall-through*?
- ▶ Apa perbedaan `while` dan `do-while`?
- ▶ Mengapa `do-while` selalu menjalankan blok setidaknya satu kali?
- ▶ Kapan `for` lebih nyaman daripada `while`?
- ▶ Apa tiga komponen utama pada `for`?
- ▶ Apa yang dimaksud dengan akumulator?
- ▶ Apa yang dimaksud dengan counter?
- ▶ Apa yang dimaksud dengan *off-by-one error*?
- ▶ Bagaimana sebuah pengulangan dapat menjadi tak hingga?
- ▶ Apa fungsi `break` di dalam loop?
- ▶ Apa fungsi `continue`?
- ▶ Mengapa dua loop bersarang sering berkaitan dengan $O(N^2)$?
- ▶ Mengapa minimum dan maksimum sebaiknya diinisialisasi dari data pertama?
- ▶ Mengapa nilai batas harus diuji secara eksplisit?
- ▶ Apa hubungan *tracing* dengan keadaan program?
- ▶ Mengapa perbandingan *floating-point* dapat memengaruhi kondisi berhenti?

4.34 Latihan

4.34.1 Klasifikasi temperatur

Buat program yang membaca temperatur dan menampilkan **DINGIN** jika

$$T < 20^{\circ}\text{C},$$

NORMAL

jika

$$20^{\circ}\text{C} \leq T \leq 30^{\circ}\text{C},$$

dan **PANAS** jika

$$T > 30^{\circ}\text{C}.$$

Uji tepat pada nilai 20 dan 30.

4.34.2 Status tekanan

Sebuah sistem memiliki rentang tekanan aman

$$95 \text{ kPa} \leq P \leq 105 \text{ kPa}.$$

Buat program yang menampilkan

LOW
NORMAL
HIGH

sesuai nilai tekanan.

4.34.3 Bilangan Reynolds

Buat program yang menghitung

$$Re = \frac{\rho v L}{\mu}$$

dan mengklasifikasikan aliran sebagai

laminar : $Re < 2300$
transition : $2300 \leq Re < 4000$
turbulent : $Re \geq 4000$

Gunakan **else if**.

4.34.4 Mode alat

Buat program menggunakan **switch** untuk menu

1. Standby
2. Measurement
3. Calibration
4. Shutdown

Jika input di luar 1, 2, 3, 4, tampilkan **Invalid mode..**

4.34.5 while

Gunakan `while` untuk mencetak

```
1
2
3
...
20
```

Kemudian modifikasi agar hanya mencetak bilangan genap.

4.34.6 for

Gunakan `for` untuk mencetak nilai

$$t = 0, 1, \dots, 10$$

dan posisi

$$x(t) = x_0 + v_0 t + \frac{1}{2} a t^2$$

dengan

$$\begin{aligned} x_0 &= 0, \\ v_0 &= 2 \text{ m/s}, \end{aligned}$$

dan

$$a = 1 \text{ m/s}^2.$$

4.34.7 Jumlah dan rata-rata

Buat program yang membaca N data real, kemudian menghitung

$$S = \sum_{i=1}^N x_i$$

dan

$$\bar{x} = \frac{S}{N}.$$

Pastikan program menolak

$$N \leq 0.$$

4.34.8 Minimum dan maksimum

Buat program yang membaca N data dan menampilkan nilai minimum serta maksimum.

Uji menggunakan data positif:

$$3, \quad 5, \quad 2, \quad 8$$

dan data negatif:

$$-3, \quad -5, \quad -2, \quad -8.$$

Pastikan algoritma benar untuk keduanya.

4.34.9 Counter ambang

Baca N data temperatur.

Hitung berapa data yang memenuhi

$$T > 30^{\circ}\text{C}.$$

Tampilkan jumlah dan persentasenya.

4.34.10 Validasi input

Gunakan `do-while` untuk membaca efisiensi η sampai memenuhi

$$0 \leq \eta \leq 1.$$

Program harus terus meminta input jika nilai tidak valid.

4.34.11 Deret

Hitung

$$S_N = \sum_{n=1}^N \frac{1}{n^2}$$

untuk beberapa nilai N .

Bandingkan dengan

$$\frac{\pi^2}{6}.$$

Gunakan

$$\pi \approx 3.141592653589793.$$

Amati bagaimana hasil berubah ketika N diperbesar.

4.34.12 Tabel Hukum Hooke

Gunakan

$$F = -kx$$

dengan

$$k = 100 \text{ N/m}.$$

Cetak tabel untuk

$$x = -0.10, -0.09, \dots, 0.10 \text{ m}.$$

Gunakan `for`.

4.34.13 Pengulangan bersarang

Buat program yang mencetak tabel perkalian 1 sampai 10.

Gunakan dua `for` bersarang.

Jelaskan mengapa jumlah operasi bertumbuh secara kuadratik terhadap ukuran tabel.

4.34.14 Mencari kesalahan

Perhatikan program berikut.

```
#include <stdio.h>

int main(void)
{
    int i = 1;

    while (i <= 10)
    {
        printf("%d\n", i);
    }

    return 0;
}
```

Jelaskan masalahnya dan perbaiki.

4.34.15 Mencari kesalahan batas

Perhatikan:

```
for (int i = 0; i <= 10; i++)
{
    printf("%d\n", i);
}
```

Berapa banyak nilai yang dicetak?

Jika tujuan sebenarnya adalah mencetak tepat 10 nilai mulai dari nol, perbaiki batasnya.

4.34.16 Monitoring temperatur

Buat program lengkap yang membaca

- ▶ jumlah pengukuran;
- ▶ ambang temperatur;
- ▶ seluruh data temperatur;

kemudian menghitung

- ▶ rata-rata;
- ▶ minimum;
- ▶ maksimum;
- ▶ jumlah data di atas ambang.

Gunakan

```
if
for
```

dan uji menggunakan

$$T = \{29.5, 30.2, 28.9, 31.1, 30.0\}.$$

4.34.17 Tantangan: berhenti dengan sentinel

Buat program yang terus membaca temperatur sampai pengguna memasukkan -999 .

Nilai -999 digunakan sebagai *sentinel* dan tidak ikut dihitung.

Setelah input selesai, tampilkan

- ▶ jumlah data valid;
- ▶ rata-rata.

Jika tidak ada data valid, jangan lakukan pembagian.

4.34.18 Tantangan: pendekatan akar

Gunakan metode sederhana dengan langkah

$$\Delta x = 0.001$$

untuk mencari pendekatan $\sqrt{2}$.

Mulai dari

$$x = 0.$$

Naikkan x sampai

$$x^2 \geq 2.$$

Bandingkan hasil dengan nilai referensi sekitar 1.41421356.

Diskusikan kekurangan metode ini.

4.35 Rangkuman

- ▶ Aliran kontrol menentukan urutan eksekusi pernyataan dalam program.
- ▶ Tiga pola dasar pemrograman terstruktur adalah

```
urutan
percabangan
pengulangan
```

- ▶ Percabangan **if** menjalankan blok jika kondisi bernilai benar.
- ▶ **if-else** memilih satu dari dua jalur.
- ▶ **if-else if-else** digunakan untuk klasifikasi dengan beberapa kondisi.